Chicago cameras crashes

```
import pandas as pd
import numpy as np
import geopandas as gpd
from shapely import wkt
import altair as alt
import matplotlib.pyplot as plt
import json
from pathlib import Path

alt.data_transformers.disable_max_rows()

REPO_ROOT = Path.cwd()
RAW_DIR = REPO_ROOT / "data" / "raw-data"
DERIVED_DIR = REPO_ROOT / "data" / "derived-data"


CRASH_FP = RAW_DIR / "Traffic_Crashes_-_Crashes_20260224.csv"
COMM_FP  = RAW_DIR / "Boundaries_-_Community_Areas_20260122.csv"
RED_FP   = RAW_DIR / "Red_Light_Camera_Locations_20260122.csv"
SPEED_FP = RAW_DIR / "Speed_Camera_Locations_20260222.csv"

crash = pd.read_csv(CRASH_FP)
comm  = pd.read_csv(COMM_FP)
redlight = pd.read_csv(RED_FP)
speed = pd.read_csv(SPEED_FP)

crash["CRASH_DATE"] = pd.to_datetime(crash["CRASH_DATE"], errors="coerce")
crash["YEAR"] = crash["CRASH_DATE"].dt.year
crash["MONTH"] = crash["CRASH_DATE"].dt.month
```

1.Data Completeness: Month Covered per Year

```
months_per_year = (
    crash.dropna(subset=["YEAR", "MONTH"])
         .groupby(["YEAR", "MONTH"])
         .size()
         .reset_index(name="n")
         .groupby("YEAR")["MONTH"]
         .nunique()
         .reset_index(name="months_covered")
         .sort_values("YEAR")
)

months_per_year
```

	YEAR	months_covered
0	2013	2
1	2014	5
2	2015	11
3	2016	12
4	2017	12
5	2018	12
6	2019	12
7	2020	12
8	2021	12
9	2022	12
10	2023	12
11	2024	12
12	2025	12
13	2026	2

```
months_chart = alt.Chart(months_per_year).mark_bar(color="#9ecae1").encode(
    x=alt.X("YEAR:O", title="Year"),
    y=alt.Y(
        "months_covered:Q",
        title="Number of Months with Data",
        scale=alt.Scale(domain=[0, 12])
    ),
    tooltip=[
        alt.Tooltip("YEAR:O", title="Year"),
        alt.Tooltip("months_covered:Q", title="Months covered")
    ]
```

```
).properties(
    title="Data Completeness: Months Covered per Year",
    width=750,
    height=400
)

months_chart
```

```
alt.Chart(...)
```

2.Annual Number of Crashed

```
annual_crashes = (
    crash.dropna(subset=["YEAR"])
         .groupby("YEAR")
         .size()
         .reset_index(name="crash_count")
         .sort_values("YEAR")
)

annual_crashes
```

	YEAR	crash_count
0	2013	2
1	2014	6
2	2015	9833
3	2016	44297
4	2017	83786
5	2018	118952
6	2019	117764
7	2020	92096
8	2021	108766
9	2022	108412
10	2023	110753
11	2024	112053
12	2025	109104
13	2026	15617

```
annual_chart = alt.Chart(annual_crashes).mark_bar(color="#43a2ca").encode(
    x=alt.X("YEAR:O", title="Year"),
    y=alt.Y("crash_count:Q", title="Annual Number of Crashes"),
    tooltip=[
        alt.Tooltip("YEAR:O", title="Year"),
        alt.Tooltip("crash_count:Q", title="Crashes")
    ]
).properties(
    title="Annual Number of Crashes",
    width=750,
    height=400
)

annual_chart
```

```
alt.Chart(...)
```

3.Average Crashed By Month (2018-2025)

```
season_df = (
    crash.dropna(subset=["YEAR", "MONTH"])
         .query("YEAR >= 2018 and YEAR <= 2025")
         .groupby(["YEAR", "MONTH"])
         .size()
         .reset_index(name="crash_count")
)

monthly_avg = (
    season_df.groupby("MONTH")["crash_count"]
             .mean()
             .reset_index(name="avg_monthly_crashes")
             .sort_values("MONTH")
)

monthly_avg
```

	MONTH	avg_monthly_crashes
0	1	8394.750
1	2	8299.875
2	3	8566.500

	MONTH	avg_monthly_crashes
3	4	8426.125
4	5	9804.250
5	6	9844.125
6	7	9788.625
7	8	9803.375
8	9	9500.375
9	10	9740.125
10	11	8735.625
11	12	8833.750

```
season_chart = alt.Chart(monthly_avg).mark_line(point=True, color="#43a2ca").encode(
    x=alt.X(
        "MONTH:O",
        title="Month",
        sort=list(range(1,13)),
        axis=alt.Axis(labelAngle=0)
    ),
    y=alt.Y("avg_monthly_crashes:Q", title="Average Monthly Crashes"),
    tooltip=[
        alt.Tooltip("MONTH:O", title="Month"),
        alt.Tooltip("avg_monthly_crashes:Q", title="Avg crashes", format=",.0f")
    ]
).properties(
    title="Seasonality: Average Crashes by Month (2018-2025)",
    width=750,
    height=400
)

season_chart
```

```
alt.Chart(...)
```

4.Average Crashes by Hour of Day (2018-2025)

```
hour_df = (
    crash.dropna(subset=["YEAR","CRASH_HOUR"])
         .query("YEAR >= 2018 and YEAR <= 2025")
         .groupby(["YEAR","CRASH_HOUR"])
         .size()
```

```
        .reset_index(name="crash_count")
)

hour_avg = (
    hour_df.groupby("CRASH_HOUR")["crash_count"]
            .mean()
            .reset_index(name="avg_crashes")
            .sort_values("CRASH_HOUR")
)

hour_avg
```

	CRASH_HOUR	avg_crashes
0	0	2473.250
1	1	2115.125
2	2	1797.625
3	3	1493.750
4	4	1307.375
5	5	1533.875
6	6	2353.750
7	7	4626.125
8	8	5814.250
9	9	4976.750
10	10	4989.375
11	11	5563.750
12	12	6388.750
13	13	6558.500
14	14	7272.125
15	15	8493.750
16	16	8376.000
17	17	8089.250
18	18	6638.000
19	19	4960.250
20	20	4062.125
21	21	3623.500
22	22	3312.625
23	23	2917.625

```
overall_avg = hour_avg["avg_crashes"].mean()
overall_avg
```

```
bars = alt.Chart(hour_avg).mark_bar(color="#43a2ca").encode(
    x=alt.X("CRASH_HOUR:O",
            title="Hour of Day (0-23)",
            axis=alt.Axis(labelAngle=0)),
    y=alt.Y("avg_crashes:Q",
            title="Average Crashes"),
    tooltip=[
        alt.Tooltip("CRASH_HOUR:O", title="Hour"),
        alt.Tooltip("avg_crashes:Q", title="Avg crashes", format=",.0f")
    ]
)

line = alt.Chart(pd.DataFrame({"y":[overall_avg]})).mark_rule(
    strokeDash=[6,6],
    color="black"
).encode(y="y:Q")

hour_chart = (bars + line).properties(
    title="Average Crashes by Hour of Day (2018-2025)",
    width=750,
    height=400
)

hour_chart
```

```
alt.LayerChart(...)
```

5.Crashes Heatmap: Hour × Day of Week (Average, 2018–2025)

```
heat_df = (
    crash.dropna(subset=["YEAR","CRASH_HOUR","CRASH_DAY_OF_WEEK"])
         .query("YEAR >= 2018 and YEAR <= 2025")
         .groupby(["YEAR","CRASH_DAY_OF_WEEK","CRASH_HOUR"])
         .size()
         .reset_index(name="crash_count")
)

heat_avg = (
    heat_df.groupby(["CRASH_DAY_OF_WEEK","CRASH_HOUR"])["crash_count"]
           .mean()
           .reset_index(name="avg_crashes")
```

```
)

heat_avg
```

	CRASH_DAY_OF_WEEK	CRASH_HOUR	avg_crashes
0	1	0	640.000
1	1	1	615.750
2	1	2	555.375
3	1	3	530.875
4	1	4	376.625
...	...	...	...
163	7	19	782.750
164	7	20	710.000
165	7	21	667.875
166	7	22	678.500
167	7	23	691.250

```
weekday_map = {
1:"Sunday",
2:"Monday",
3:"Tuesday",
4:"Wednesday",
5:"Thursday",
6:"Friday",
7:"Saturday"
}

heat_avg["weekday"] = heat_avg["CRASH_DAY_OF_WEEK"].map(weekday_map)

heat_chart = alt.Chart(heat_avg).mark_rect().encode(
    x=alt.X("CRASH_HOUR:O",
            title="Hour of Day (0-23)"),

    y=alt.Y("weekday:O",
            title="Day of Week",
            sort=["Monday","Tuesday","Wednesday","Thursday","Friday","Saturday","Sunday"]),

    color=alt.Color(
        "avg_crashes:Q",
        title="Avg Crashes",
```

```
        scale=alt.Scale(
            range=["#e0f3db","#a8ddb5","#43a2ca"]
        )
    ),

    tooltip=[
        alt.Tooltip("weekday:N", title="Day"),
        alt.Tooltip("CRASH_HOUR:O", title="Hour"),
        alt.Tooltip("avg_crashes:Q", title="Avg crashes", format=",.0f")
    ]
).properties(
    title="Crashes Heatmap: Hour × Day of Week (Average, 2018-2025)",
    width=750,
    height=350
)

heat_chart
```

```
alt.Chart(...)
```

6.Crash Severity Distribution

```
severity_df = (
    crash.dropna(subset=["YEAR","MOST_SEVERE_INJURY"])
         .query("YEAR >= 2018 and YEAR <= 2025")
)

severity_share = (
    severity_df["MOST_SEVERE_INJURY"]
    .value_counts(normalize=True)
    .mul(100)
    .reset_index()
)

severity_share.columns = ["severity","share"]
severity_share
```

	severity	share
0	NO INDICATION OF INJURY	84.868371
1	NONINCAPACITATING INJURY	8.523051

	severity	share
2	REPORTED, NOT EVIDENT	4.780074
3	INCAPACITATING INJURY	1.716621
4	FATAL	0.111884

```
order = [
"NO INDICATION OF INJURY",
"REPORTED, NOT EVIDENT",
"NONINCAPACITATING INJURY",
"INCAPACITATING INJURY",
"FATAL"
]

severity_share["severity"] = pd.Categorical(
    severity_share["severity"],
    categories=order,
    ordered=True
)

severity_share = severity_share.sort_values("severity")

severity_chart = alt.Chart(severity_share).mark_bar(
    color="#43a2ca"
).encode(
    y=alt.Y(
        "severity:N",
        sort=order,
        title="Most Severe Injury"
    ),

    x=alt.X(
        "share:Q",
        title="Share of Crashes (%)"
    ),

    tooltip=[
        alt.Tooltip("severity:N", title="Severity"),
        alt.Tooltip("share:Q", title="Share (%)", format=".1f")
    ]
).properties(
    title="Crash Severity Distribution (2018-2025)",
```

```
    width=750,
    height=350
)

severity_chart
text = severity_chart.mark_text(
    align="left",
    baseline="middle",
    dx=5
).encode(
    text=alt.Text("share:Q", format=".1f")
)

severity_chart + text
```

```
alt.LayerChart(...)
```

7.Top Roadway Surface Conditions in Crashes

```
df = crash.query("YEAR >= 2018 and YEAR <= 2025").copy()
col = "ROADWAY_SURFACE_COND"

top_n = 10
counts = df[col].fillna("UNKNOWN").astype(str).value_counts()

top = counts.head(top_n)
other = counts.iloc[top_n:].sum()

surf = pd.concat([top, pd.Series({"OTHER": other})]).reset_index()
surf.columns = ["surface", "n"]
surf["share_pct"] = surf["n"] / surf["n"].sum() * 100
surf
```

	surface	n	share_pct
0	DRY	642340	73.167787
1	WET	111633	12.715913
2	UNKNOWN	86742	9.880624
3	SNOW OR SLUSH	28447	3.240346
4	ICE	6052	0.689372
5	OTHER	2398	0.273152

	surface	n	share_pct
6	SAND, MUD, DIRT	288	0.032806
7	OTHER	0	0.000000

```
base = alt.Chart(surf).mark_bar(color="#43a2ca").encode(
    y=alt.Y("surface:N", sort="-x", title=None),
    x=alt.X("n:Q", title="Crashes"),
    tooltip=["surface", alt.Tooltip("n:Q", format=",.0f"), alt.Tooltip("share_pct:Q", format=
).properties(title="Top Roadway Surface Conditions in Crashes", width=750, height=360)
base
```

```
alt.Chart(...)
```

8.Lightning Conditions in Crashes

```
df = crash.query("YEAR >= 2018 and YEAR <= 2025").copy()
col = "LIGHTING_CONDITION"

top_n = 10
counts = df[col].fillna("UNKNOWN").astype(str).value_counts()

top = counts.head(top_n)
other = counts.iloc[top_n:].sum()

light = pd.concat([top, pd.Series({"OTHER": other})]).reset_index()
light.columns = ["lighting", "n"]
light["share_pct"] = light["n"] / light["n"].sum() * 100
light
```

	lighting	n	share_pct
0	DAYLIGHT	560709	63.869347
1	DARKNESS, LIGHTED ROAD	193980	22.095911
2	UNKNOWN	44350	5.051828
3	DARKNESS	39890	4.543798
4	DUSK	24299	2.767855
5	DAWN	14672	1.671261
6	OTHER	0	0.000000

```
base = alt.Chart(light).mark_bar(color="#a8ddb5").encode(
    y=alt.Y("lighting:N", sort="-x", title=None),
    x=alt.X("n:Q", title="Crashes"),
    tooltip=["lighting", alt.Tooltip("n:Q", format=",.0f"), alt.Tooltip("share_pct:Q", forma
).properties(title="Lighting", width=750, height=320)

base
```

```
alt.Chart(...)
```

9.Primary contributory causes

```
df = crash.query("YEAR >= 2018 and YEAR <= 2025").copy()
col = "PRIM_CONTRIBUTORY_CAUSE"

top_n = 10
counts = df[col].fillna("UNKNOWN").astype(str).value_counts()

top = counts.head(top_n)
other = counts.iloc[top_n:].sum()

cause = pd.concat([top, pd.Series({"OTHER": other})]).reset_index()
cause.columns = ["cause", "n"]
cause["share_pct"] = cause["n"] / cause["n"].sum() * 100
cause
```

	cause	n	share_pct
0	UNABLE TO DETERMINE	348587	39.706914
1	FAILING TO YIELD RIGHT-OF-WAY	98164	11.181684
2	FOLLOWING TOO CLOSELY	80164	9.131336
3	NOT APPLICABLE	45491	5.181797
4	IMPROPER OVERTAKING/PASSING	44346	5.051373
5	FAILING TO REDUCE SPEED TO AVOID CRASH	37901	4.317234
6	IMPROPER BACKING	31763	3.618066
7	DRIVING SKILLS/KNOWLEDGE/EXPERIENCE	30730	3.500399
8	IMPROPER LANE USAGE	30206	3.440711
9	IMPROPER TURNING/NO SIGNAL	29706	3.383757
10	OTHER	100842	11.486730

```
base = alt.Chart(cause).mark_bar(color="#43a2ca").encode(
    y=alt.Y("cause:N", sort="-x", title=None),
    x=alt.X("n:Q", title="Crashes"),
    tooltip=["cause", alt.Tooltip("n:Q", format=",.0f"), alt.Tooltip("share_pct:Q", format="
).properties(title="Primary Cause", width=750, height=420)
base
```

```
alt.Chart(...)
```

10.eatmap of total injuries Year × Month

```
inj = (
    crash.query("YEAR >= 2018 and YEAR <= 2025")
         .dropna(subset=["YEAR","MONTH"])
         .groupby(["YEAR","MONTH"])["INJURIES_TOTAL"]
         .sum()
         .reset_index(name="total_injuries")
)

inj.head()
```

	YEAR	MONTH	total_injuries
0	2018	1	1786.0
1	2018	2	1297.0
2	2018	3	1647.0
3	2018	4	1794.0
4	2018	5	2161.0

```
inj_heat = alt.Chart(inj).mark_rect().encode(
    x=alt.X("MONTH:O", title="Month", sort=list(range(1,13)), axis=alt.Axis(labelAngle=0)),
    y=alt.Y("YEAR:O", title="Year", sort="ascending"),
    color=alt.Color(
        "total_injuries:Q",
        title="Injuries",
        scale=alt.Scale(range=["#e0f3db", "#a8ddb5", "#43a2ca"])
    ),
    tooltip=["YEAR:O","MONTH:O", alt.Tooltip("total_injuries:Q", format=",.0f")]
).properties(title="Injuries Heatmap", width=750, height=320)

inj_heat
```

```
alt.Chart(...)
```

11.Total injuries over time

```
df = crash.query("YEAR >= 2018 and YEAR <= 2025").copy()
df = df.dropna(subset=["CRASH_DATE"])

df["year_month"] = df["CRASH_DATE"].dt.to_period("M").dt.to_timestamp()

inj_month = (
    df.groupby("year_month")["INJURIES_TOTAL"]
      .sum()
      .reset_index(name="total_injuries")
      .sort_values("year_month")
)

inj_line = alt.Chart(inj_month).mark_line(color="#43a2ca").encode(
    x=alt.X("year_month:T", title="Month"),
    y=alt.Y("total_injuries:Q", title="Total injuries"),
    tooltip=[alt.Tooltip("year_month:T", title="Month"),
             alt.Tooltip("total_injuries:Q", format=",.0f")]
).properties(title="Injuries Over Time", width=750, height=320)

inj_line
```

```
alt.Chart(...)
```

12.Fatal rate by first crash type

```
df = crash.query("YEAR >= 2018 and YEAR <= 2025").copy()

df["fatal_flag"] = df["INJURIES_FATAL"].fillna(0).astype(float) > 0

tmp = (
    df.dropna(subset=["FIRST_CRASH_TYPE"])
      .groupby("FIRST_CRASH_TYPE")
      .agg(
          total=("FIRST_CRASH_TYPE","size"),
          fatal=("fatal_flag","sum")
      )
      .reset_index()
```

```
)

tmp["fatal_rate"] = tmp["fatal"] / tmp["total"]

min_total = 1000
top_k = 15
fatal_type = (
    tmp.query("total >= @min_total")
       .sort_values("fatal_rate", ascending=False)
       .head(top_k)
)

fatal_bar = alt.Chart(fatal_type).mark_bar().encode(
    x=alt.X("FIRST_CRASH_TYPE:N", sort="-y", title=None),
    y=alt.Y("fatal_rate:Q", title="Fatal rate"),
    color=alt.condition(
        alt.datum.fatal_rate >= fatal_type["fatal_rate"].median(),
        alt.value("#43a2ca"),
        alt.value("#a8ddb5")
    ),
    tooltip=["FIRST_CRASH_TYPE",
             alt.Tooltip("total:Q", format=",.0f"),
             alt.Tooltip("fatal:Q", format=",.0f"),
             alt.Tooltip("fatal_rate:Q", format=".4f")]
).properties(title="Fatal Rate by Crash Type", width=750, height=320)

fatal_bar
```

```
alt.Chart(...)
```

13.Annual fatal crashes

```
df = crash.query("YEAR >= 2018 and YEAR <= 2025").copy()

df["fatal_flag"] = df["INJURIES_FATAL"].fillna(0).astype(float) > 0

year_fatal = (
    df.groupby("YEAR")
      .agg(
          total_crashes=("fatal_flag", "size"),
          fatal_crashes=("fatal_flag", "sum")
```

```
      )
      .reset_index()
      .sort_values("YEAR")
)

year_fatal["fatal_share_pct"] = year_fatal["fatal_crashes"] / year_fatal["total_crashes"] * 1
year_fatal
```

	YEAR	total_crashes	fatal_crashes	fatal_share_pct
0	2018	118952	114	0.095837
1	2019	117764	102	0.086614
2	2020	92096	134	0.145500
3	2021	108766	156	0.143427
4	2022	108412	135	0.124525
5	2023	110753	143	0.129116
6	2024	112053	111	0.099060
7	2025	109104	85	0.077907

```
fatal_count_chart = alt.Chart(year_fatal).mark_line(point=True, color="#43a2ca").encode(
    x=alt.X("YEAR:O", title="Year"),
    y=alt.Y("fatal_crashes:Q", title="Fatal crashes"),
    tooltip=["YEAR", "fatal_crashes", alt.Tooltip("total_crashes:Q", format=",")]
).properties(
    title="Annual Fatal Crashes",
    width=600,
    height=360
)

fatal_count_chart
```

```
alt.Chart(...)
```

14.Annual fatal share

```
fatal_share_chart = alt.Chart(year_fatal).mark_line(point=True, color="#43a2ca").encode(
    x=alt.X("YEAR:O", title="Year"),
    y=alt.Y("fatal_share_pct:Q", title="Fatal share (%)"),
    tooltip=["YEAR", alt.Tooltip("fatal_share_pct:Q", format=".3f"),
             alt.Tooltip("fatal_crashes:Q", format=",.0f"),
```

```
              alt.Tooltip("total_crashes:Q", format=",.0f")]
).properties(
    title="Fatal Share of Crashes",
    width=600,
    height=360
)

fatal_share_chart
```

```
alt.Chart(...)
```

15.Fatal Crash Rate by Posted Speed Limit

```
df = crash.query("YEAR >= 2018 and YEAR <= 2025").copy()

df["POSTED_SPEED_LIMIT"] = pd.to_numeric(df["POSTED_SPEED_LIMIT"], errors="coerce")
df = df.dropna(subset=["POSTED_SPEED_LIMIT"])
df = df[(df["POSTED_SPEED_LIMIT"] >= 0) & (df["POSTED_SPEED_LIMIT"] <= 70)]

df["fatal_flag"] = df["INJURIES_FATAL"].fillna(0).astype(float) > 0

bins = [0,20,25,30,35,40,45,50,55,60,65,70]
labels = ["0-20","21-25","26-30","31-35","36-40","41-45","46-50","51-55","56-60","61-65","66-

df["speed_bin"] = pd.cut(df["POSTED_SPEED_LIMIT"], bins=bins, labels=labels, include_lowest=

speed_stats = (
    df.dropna(subset=["speed_bin"])
      .groupby("speed_bin")
      .agg(total=("fatal_flag","size"), fatal=("fatal_flag","sum"))
      .reset_index()
)

speed_stats["fatal_rate"] = speed_stats["fatal"] / speed_stats["total"]

low_bins  = {"0-20","21-25","26-30"}
mid_bins  = {"31-35","51-55"}
high_bins = {"36-40","41-45","46-50"}

def pick_color(b):
    if b in low_bins:  return "#e0f3db"
```

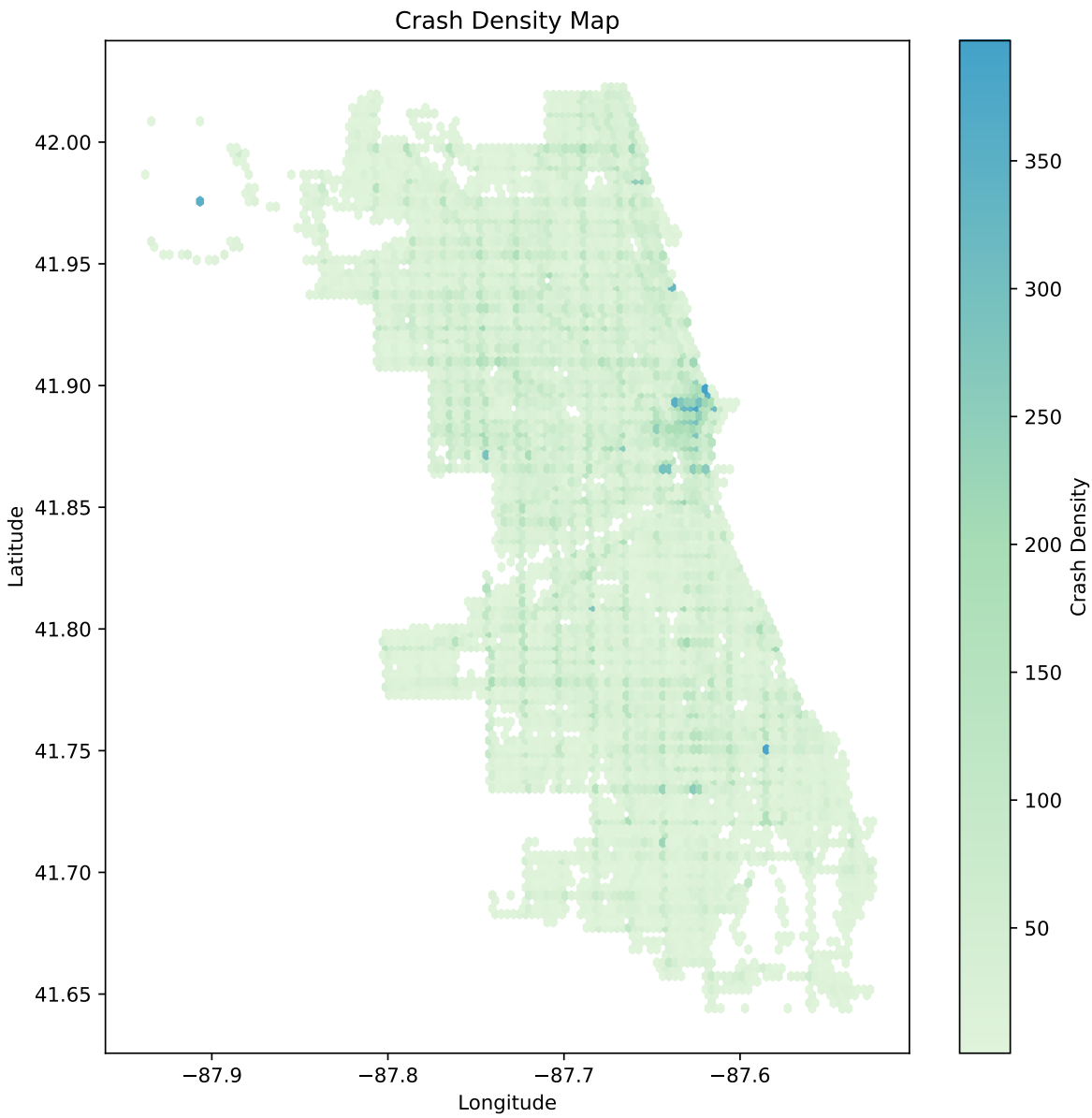
```
    if b in high_bins: return "#43a2ca"
    return "#a8ddb5"

speed_stats["bar_color"] = speed_stats["speed_bin"].astype(str).map(pick_color)

bars = alt.Chart(speed_stats).mark_bar(
    size=50
).encode(
    x=alt.X(
        "speed_bin:O",
        title="Posted Speed Limit (mph)",
        sort=labels,
        axis=alt.Axis(labelAngle=0)
    ),

    y=alt.Y(
        "fatal_rate:Q",
        title="Fatal Rate"
    ),

    color=alt.Color("bar_color:N", scale=None, legend=None),

    tooltip=[
        alt.Tooltip("speed_bin:N", title="Speed"),
        alt.Tooltip("fatal_rate:Q", format=".4f"),
        alt.Tooltip("total:Q", format=",")
    ]
)

line = alt.Chart(speed_stats).mark_line(
    color="#2c5c85",
    point=True,
    strokeWidth=1
).encode(
    x=alt.X("speed_bin:O", sort=labels),
    y="fatal_rate:Q"
)

chart = (bars + line).properties(
    title="Fatal Crash Rate by Posted Speed Limit",
    width=800,
    height=380
```

```
).configure_view(
    stroke="black",
    strokeWidth=1.5
)

chart
```

```
alt.LayerChart(...)
```

16.Crash Density Map

```
from matplotlib.colors import LinearSegmentedColormap

colors = ["#e0f3db", "#a8ddb5", "#43a2ca"]
cmap_gnbu = LinearSegmentedColormap.from_list("gnbu_custom", colors)

m = crash.copy()
m = m[(m["YEAR"] >= 2018) & (m["YEAR"] <= 2025)]
m = m.dropna(subset=["LATITUDE", "LONGITUDE"])

m = m[(m["LATITUDE"].between(41.6, 42.1)) &
      (m["LONGITUDE"].between(-88.0, -87.3))]

if len(m) > 200000:
    m = m.sample(200000, random_state=0)

plt.figure(figsize=(8, 8))

hb = plt.hexbin(
    m["LONGITUDE"],
    m["LATITUDE"],
    gridsize=120,
    mincnt=1,
    cmap=cmap_gnbu
)

plt.colorbar(hb, label="Crash Density")
plt.title("Crash Density Map")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.tight_layout()
plt.show()
```



Clean speed data

```
speed["GO-LIVE DATE"] = pd.to_datetime(speed["GO-LIVE DATE"], errors="coerce")
speed["year"] = speed["GO-LIVE DATE"].dt.year

speed["LATITUDE"] = pd.to_numeric(speed["LATITUDE"], errors="coerce")
speed["LONGITUDE"] = pd.to_numeric(speed["LONGITUDE"], errors="coerce")
speed = speed.dropna(subset=["LATITUDE", "LONGITUDE"])
```

```
speed[["GO-LIVE DATE","year","LATITUDE","LONGITUDE"]].head()
```

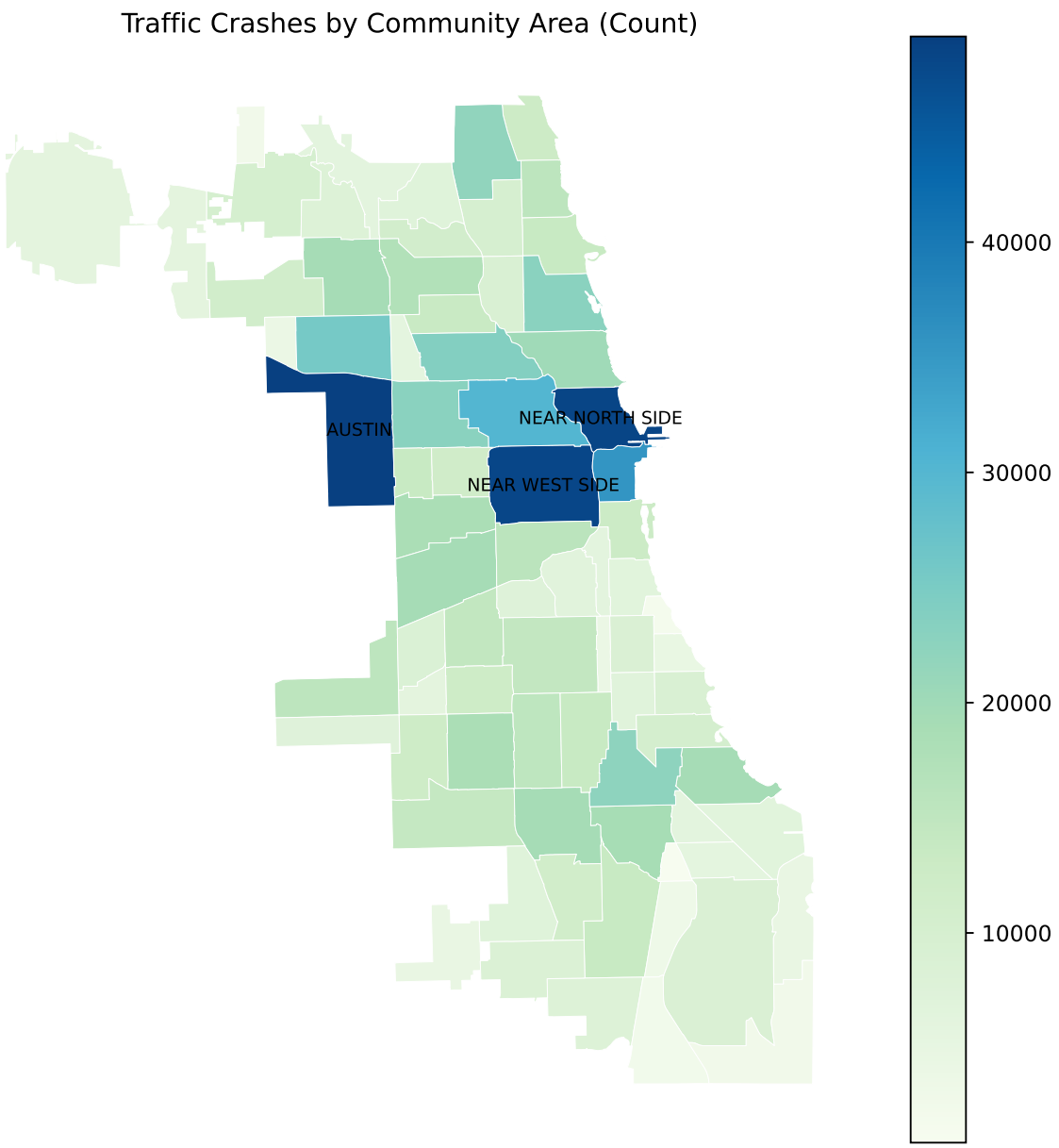
	GO-LIVE DATE	year	LATITUDE	LONGITUDE
0	2013-09-06	2013	41.7644	-87.7097
1	2013-12-21	2013	41.9236	-87.7825
2	2013-12-21	2013	41.9238	-87.7826
3	2013-12-12	2013	41.9239	-87.7639
4	2013-12-12	2013	41.9241	-87.7630

We built three plots to compare the geographic distribution of crashes and traffic cameras across Chicago community areas. First, this heatmap shows total crash counts by community area. We set up a function to read the community boundaries data and convert it to a GeoDataFrame. It also has an optional plotting feature to visualize the boundaries.

```
def community_boundaries(plot=False):

    boundaries_gdf = gpd.GeoDataFrame(
        comm,
        geometry=gpd.GeoSeries.from_wkt(comm["the_geom"]),
        crs="EPSG:4326"
    )

    if plot:
        fig, ax = plt.subplots(figsize=(8, 8))
        boundaries_gdf.plot(ax=ax, edgecolor="black", linewidth=0.6, facecolor="none")
        ax.set_axis_off()
        plt.show()

    return boundaries_gdf
```

```
boundaries_gdf = community_boundaries()

crashes_clean = crash.dropna(subset=["LATITUDE", "LONGITUDE"]).copy()

crashes_gdf = gpd.GeoDataFrame(
    crashes_clean,
    geometry=gpd.points_from_xy(crashes_clean["LONGITUDE"], crashes_clean["LATITUDE"]),
    crs="EPSG:4326"
```

```
)

boundaries_small = boundaries_gdf[["COMMUNITY", "geometry"]].copy()
joined = gpd.sjoin(crashes_gdf, boundaries_small, how="inner", predicate="within")

crash_ct = (
    joined.groupby("COMMUNITY")
    .size()
    .rename("crash_count")
    .reset_index()
)

areas_plot = boundaries_gdf.merge(crash_ct, on="COMMUNITY", how="left")
areas_plot["crash_count"] = areas_plot["crash_count"].fillna(0).astype(int)

fig, ax = plt.subplots(figsize=(9, 9))
areas_plot.plot(
    column="crash_count",
    cmap="GnBu",
    ax=ax,
    legend=True,
    edgecolor="white",
    linewidth=0.3
)

top3_geoms = areas_plot.sort_values("crash_count", ascending=False).head(3).copy()
top3_geoms["pt"] = top3_geoms.geometry.representative_point()

for _, r in top3_geoms.iterrows():
    ax.text(
        r["pt"].x, r["pt"].y,
        r["COMMUNITY"],
        fontsize=8,
        ha="center",
        va="center"
    )

ax.set_axis_off()
ax.set_title("Traffic Crashes by Community Area (Count)")
plt.show()
```



Second, we used the same way to map speed cameras. Speed cameras are spatially concentrated. Several neighborhoods exhibit substantially higher deployment intensity.

```
comm_gdf = gpd.GeoDataFrame(
    comm.copy(),
    geometry=comm["the_geom"].apply(wkt.loads),
    crs="EPSG:4326"
)
```

```python
speed_gdf = gpd.GeoDataFrame(
    speed.copy(),
    geometry=gpd.points_from_xy(speed["LONGITUDE"], speed["LATITUDE"]),
    crs="EPSG:4326"
)

speed_comm = gpd.sjoin(speed_gdf, comm_gdf, how="left", predicate="within")

camera_counts = (
    speed_comm.groupby("COMMUNITY")
    .size()
    .reset_index(name="num_cameras")
)

comm_gdf = comm_gdf.merge(camera_counts, on="COMMUNITY", how="left")
comm_gdf["num_cameras"] = comm_gdf["num_cameras"].fillna(0).astype(int)
```

```python
PALETTE_MAIN = ["#e0f3db", "#a8ddb5", "#43a2ca"]
PALETTE_CONTRAST = ["#fee8c8", "#fdbb84", "#e34a33"]
```

```python
def map_speed_cameras(comm_gdf, speed_gdf, palette, title):
    comm_geojson = json.loads(comm_gdf.to_json())
    points_df = speed_gdf[["LONGITUDE","LATITUDE"]].dropna().copy()

    base = (
        alt.Chart(alt.Data(values=comm_geojson["features"]))
        .mark_geoshape(stroke="white", strokeWidth=0.5)
        .encode(
            color=alt.Color(
                "properties.num_cameras:Q",
                scale=alt.Scale(range=palette),
                legend=alt.Legend(title="Speed cameras")
            ),
            tooltip=[
                alt.Tooltip("properties.COMMUNITY:N", title="Community"),
                alt.Tooltip("properties.num_cameras:Q", title="Num cameras"),
            ],
        )
        .project(type="mercator")
        .properties(width=650, height=650, title=title)
    )
```

```
    pts = (
        alt.Chart(points_df)
        .mark_circle(size=12, opacity=0.5, color=palette[-1])
        .encode(longitude="LONGITUDE:Q", latitude="LATITUDE:Q")
    )

    return base + pts

map_speed_cameras(comm_gdf, speed_gdf, PALETTE_MAIN, "Speed cameras by community")
```

```
alt.LayerChart(...)
```

clean red light data

```
redlight["GO LIVE DATE"] = pd.to_datetime(redlight["GO LIVE DATE"], errors="coerce")
redlight["year"] = redlight["GO LIVE DATE"].dt.year

redlight["LATITUDE"] = pd.to_numeric(redlight["LATITUDE"], errors="coerce")
redlight["LONGITUDE"] = pd.to_numeric(redlight["LONGITUDE"], errors="coerce")
redlight = redlight.dropna(subset=["LATITUDE", "LONGITUDE"]).copy()

redlight[["GO LIVE DATE", "year", "LATITUDE", "LONGITUDE"]].head()
```

	GO LIVE DATE	year	LATITUDE	LONGITUDE
0	2007-10-03	2007	41.92431	-87.76610
1	2007-05-06	2007	41.89543	-87.68670
2	2008-11-13	2008	41.92356	-87.78583
3	2009-08-17	2009	41.93136	-87.76588
4	2009-06-07	2009	41.94665	-87.68870

Third, we map red light cameras.

```
red_gdf = gpd.GeoDataFrame(
    redlight.copy(),
    geometry=gpd.points_from_xy(redlight["LONGITUDE"], redlight["LATITUDE"]),
    crs="EPSG:4326"
)

red_comm = gpd.sjoin(red_gdf, comm_gdf, how="left", predicate="within")
```

```
red_counts = (
    red_comm.groupby("COMMUNITY")
    .size()
    .reset_index(name="num_red_cameras")
)

comm_red_gdf = comm_gdf.merge(red_counts, on="COMMUNITY", how="left")
comm_red_gdf["num_red_cameras"] = comm_red_gdf["num_red_cameras"].fillna(0).astype(int)

red_counts.sort_values("num_red_cameras", ascending=False).head(10)
```

	COMMUNITY	num_red_cameras
27	LOGAN SQUARE	17
40	PORTAGE PARK	17
6	BELMONT CRAGIN	16
4	AUSTIN	14
34	NEAR WEST SIDE	12
21	HUMBOLDT PARK	12
33	NEAR NORTH SIDE	11
55	WEST RIDGE	10
22	IRVING PARK	10
56	WEST TOWN	9

```
def map_redlight_cameras(comm_gdf_in, red_gdf_in, palette, title):
    comm_geojson = json.loads(comm_gdf_in.to_json())
    points_df = red_gdf_in[["LONGITUDE","LATITUDE"]].dropna().copy()

    base = (
        alt.Chart(alt.Data(values=comm_geojson["features"]))
        .mark_geoshape(stroke="white", strokeWidth=0.5)
        .encode(
            color=alt.Color(
                "properties.num_red_cameras:Q",
                scale=alt.Scale(range=palette),
                legend=alt.Legend(title="Red light cameras")
            ),
            tooltip=[
                alt.Tooltip("properties.COMMUNITY:N", title="Community"),
                alt.Tooltip("properties.num_red_cameras:Q", title="Num red light cameras"),
            ],
```

```
        )
        .project(type="mercator")
        .properties(width=650, height=650, title=title)
    )

    pts = (
        alt.Chart(points_df)
        .mark_circle(size=12, opacity=0.5, color=palette[-1], stroke="white", strokeWidth=0.2
        .encode(longitude="LONGITUDE:Q", latitude="LATITUDE:Q")
    )

    return base + pts

map_redlight_cameras(comm_red_gdf, red_gdf, PALETTE_CONTRAST, "Red light cameras by communit
```

```
alt.LayerChart(...)
```

Timing of speed camera deployment

```
speed["year"] = speed["GO-LIVE DATE"].dt.year

alt.Chart(speed).mark_bar().encode(
    x=alt.X("year:O", title="Go Live Year"),
    y=alt.Y("count():Q", title="Number of Cameras"),
    tooltip=["year", "count()"]
).properties(
    width=700,
    height=400,
    title="Speed Camera Deployment by Year"
)
```

```
alt.Chart(...)
```

Speed camera deployment in Chicago occurred in distinct waves. The first major rollout took place between 2013 and 2014, accounting for over half of all installations. After a period of relative stability, a second expansion began in 2023 and intensified in 2025. This staggered adoption pattern implies that treatment timing varies across locations, requiring an event-time framework for causal analysis.

clean crash data

```python
import pandas as pd

cols = pd.read_csv(CRASH_FP, nrows=0).columns.tolist()
len(cols)
crash.columns
```

```
Index(['CRASH_RECORD_ID', 'CRASH_DATE_EST_I', 'CRASH_DATE',
       'POSTED_SPEED_LIMIT', 'TRAFFIC_CONTROL_DEVICE', 'DEVICE_CONDITION',
       'WEATHER_CONDITION', 'LIGHTING_CONDITION', 'FIRST_CRASH_TYPE',
       'TRAFFICWAY_TYPE', 'LANE_CNT', 'ALIGNMENT', 'ROADWAY_SURFACE_COND',
       'ROAD_DEFECT', 'REPORT_TYPE', 'CRASH_TYPE', 'INTERSECTION_RELATED_I',
       'NOT_RIGHT_OF_WAY_I', 'HIT_AND_RUN_I', 'DAMAGE', 'DATE_POLICE_NOTIFIED',
       'PRIM_CONTRIBUTORY_CAUSE', 'SEC_CONTRIBUTORY_CAUSE', 'STREET_NO',
       'STREET_DIRECTION', 'STREET_NAME', 'BEAT_OF_OCCURRENCE',
       'PHOTOS_TAKEN_I', 'STATEMENTS_TAKEN_I', 'DOORING_I', 'WORK_ZONE_I',
       'WORK_ZONE_TYPE', 'WORKERS_PRESENT_I', 'NUM_UNITS',
       'MOST_SEVERE_INJURY', 'INJURIES_TOTAL', 'INJURIES_FATAL',
       'INJURIES_INCAPACITATING', 'INJURIES_NON_INCAPACITATING',
       'INJURIES_REPORTED_NOT_EVIDENT', 'INJURIES_NO_INDICATION',
       'INJURIES_UNKNOWN', 'CRASH_HOUR', 'CRASH_DAY_OF_WEEK', 'CRASH_MONTH',
       'LATITUDE', 'LONGITUDE', 'LOCATION', 'YEAR', 'MONTH'],
      dtype='object')
```

```python
crash["CRASH_DATE"] = pd.to_datetime(crash["CRASH_DATE"], errors="coerce")
crash["CRASH_DATE"].min()
crash["CRASH_DATE"].max()
```

```
Timestamp('2026-02-24 00:15:00')
```

Crash data began since 2013-03 but the camera that came to live in 2013 did not have an enough "pre period".

Event study-speed camera

```python
speed_2nd = speed[speed["GO-LIVE DATE"] >= "2023-01-01"].copy()
len(speed_2nd)
speed_2nd.columns
```

```
Index(['ID', 'LOCATION ID', 'ADDRESS', 'FIRST APPROACH', 'SECOND APPROACH',
       'GO-LIVE DATE', 'LATITUDE', 'LONGITUDE', 'LOCATION', 'year'],
      dtype='object')
```

set a 200m buffer

```
speed_2nd_gdf = gpd.GeoDataFrame(
    speed_2nd,
    geometry=gpd.points_from_xy(speed_2nd["LONGITUDE"],
                                speed_2nd["LATITUDE"]),
    crs="EPSG:4326"
)

# meter coordinates
speed_2nd_gdf = speed_2nd_gdf.to_crs(epsg=26971)

# 200m buffer
speed_2nd_gdf["geometry"] = speed_2nd_gdf.buffer(200)
```

crash data geodataframe

```
crash_gdf = gpd.GeoDataFrame(
    crash,
    geometry=gpd.points_from_xy(crash["LONGITUDE"],
                                crash["LATITUDE"]),
    crs="EPSG:4326"
)

crash_gdf = crash_gdf.to_crs(epsg=26971)
```

spatial join

```
crash_camera = gpd.sjoin(
    crash_gdf,
    speed_2nd_gdf[["LOCATION ID", "GO-LIVE DATE", "geometry"]],
    how="inner",
    predicate="within"
)
```

```
crash_camera["crash_month"] = crash_camera["CRASH_DATE"].dt.to_period("M")
crash_camera["go_live_month"] = crash_camera["GO-LIVE DATE"].dt.to_period("M")

crash_camera["event_time"] = (
    crash_camera["crash_month"] -
    crash_camera["go_live_month"]
```

```
).apply(lambda x: x.n)

crash_camera = crash_camera[
    (crash_camera["event_time"] >= -24) &
    (crash_camera["event_time"] <= 24)
]

panel = (
    crash_camera
    .groupby(["LOCATION ID", "event_time"])
    .size()
    .reset_index(name="crash_count")
)

event_avg = (
    panel
    .groupby("event_time")["crash_count"]
    .mean()
    .reset_index()
)
```

```
chart = alt.Chart(event_avg).mark_line(point=True).encode(
    x=alt.X(
        "event_time:Q",
        title="Months Since Installation"
    ),
    y=alt.Y(
        "crash_count:Q",
        title="Average Monthly Crashes per Speed Camera"
    )
).properties(
    width=700,
    height=400,
    title="Event Study: Speed Cameras (Install Year   2023, ±24 Months)"
)

vline = alt.Chart(
    pd.DataFrame({"x": [0]})
).mark_rule(
    color="red",
    strokeDash=[6,4]
).encode(
```

```
    x="x:Q"
)

chart + vline
```

```
alt.LayerChart(...)
```

We evaluate the impact of speed camera installation on traffic crashes using a staggered event study design. By spatially matching crash incidents within 200 meters of each camera and aligning time relative to installation month, we examine crash trends 24 months before and after deployment. The pre-treatment trend appears stable, suggesting no evidence of anticipatory decline. Following installation, average monthly crashes per camera decline by approximately 15–25%, indicating that speed cameras are associated with a meaningful reduction in local crash frequency.

Clean red light data

```
redlight["GO LIVE DATE"] = pd.to_datetime(redlight["GO LIVE DATE"], errors="coerce")

redlight["GO LIVE DATE"].min(), redlight["GO LIVE DATE"].max()
```

```
(Timestamp('2004-02-24 00:00:00'), Timestamp('2018-02-18 00:00:00'))
```

Since there are FIRST/SECOND/THIRD APPROACH the same INTERSECTION may appear repeatedly.

```
redlight["INTERSECTION"].nunique(), len(redlight)
```

```
(264, 300)
```

For the same interaction, take the earliest GO LIVE DATE.

```
redlight_intersection = (
    redlight.sort_values("GO LIVE DATE")
      .groupby("INTERSECTION", as_index=False)
      .agg({
          "GO LIVE DATE": "min",
          "LATITUDE": "first",
          "LONGITUDE": "first",
          "LOCATION": "first"
```

```
      })
)

redlight_intersection.shape, redlight_intersection.head()

redlight_intersection["install_year"] = redlight_intersection["GO LIVE DATE"].dt.year
redlight_intersection["install_year"].value_counts().sort_index()
```

```
install_year
2004    35
2005     4
2006    12
2007    50
2008    82
2009    56
2010    14
2011     1
2017     2
2018     8
Name: count, dtype: int64
```

Timing of red light camera deployment

```
df = redlight_intersection.copy()
df["GO LIVE DATE"] = pd.to_datetime(df["GO LIVE DATE"], errors="coerce")
df = df.dropna(subset=["GO LIVE DATE"])

df["install_year"] = df["GO LIVE DATE"].dt.year

year_counts = (
    df.groupby("install_year")
      .size()
      .reset_index(name="num_intersections")
)

chart_year = alt.Chart(year_counts).mark_bar().encode(
    x=alt.X("install_year:O", title="Go-Live Year"),
    y=alt.Y("num_intersections:Q", title="Number of Intersections"),
    tooltip=[
        alt.Tooltip("install_year:O", title="Year"),
        alt.Tooltip("num_intersections:Q", title="# Intersections")
```

```
    ]
).properties(
    width=700,
    height=400,
    title="Red Light Camera Deployment by Year (Intersections)"
)

chart_year
```

```
alt.Chart(...)
```

```
redlight_gdf = gpd.GeoDataFrame(
    redlight_intersection,
    geometry=gpd.points_from_xy(redlight_intersection["LONGITUDE"], redlight_intersection["LA
    crs="EPSG:4326"
)
redlight_gdf.head()
```

	INTERSECTION	GO LIVE DATE	LATITUDE	LONGITUDE	LOCATION	inst
0	0 N. Ashland Ave	2004-03-04	41.88122	-87.66663	(41.88122, -87.66663)	200
1	0 North Hamlin Boulevard	2009-04-28	41.88050	-87.72058	(41.8805, -87.72058)	200
2	1 East 63rd St	2006-01-17	41.78008	-87.62507	(41.78008, -87.62507)	200
3	1 East 75th Street	2009-08-27	41.75823	-87.62424	(41.75823, -87.62424)	200
4	1 East 79th Street	2009-06-25	41.75105	-87.62419	(41.75105, -87.62419)	200

Event study - red light camera

```
cutoff_year = 2018

redlight_intersection_sub = redlight_intersection[
    redlight_intersection["GO LIVE DATE"].dt.year >= cutoff_year
].copy()

redlight_gdf_sub = gpd.GeoDataFrame(
    redlight_intersection_sub,
    geometry=gpd.points_from_xy(redlight_intersection_sub["LONGITUDE"],
                                redlight_intersection_sub["LATITUDE"]),
    crs="EPSG:4326"
)
```

```
redlight_proj_sub = redlight_gdf_sub.to_crs(epsg=26971)
redlight_proj_sub["buffer_200m"] = redlight_proj_sub.geometry.buffer(200)
buffer_gdf_sub = redlight_proj_sub.set_geometry("buffer_200m")
```

```
crash_camera = gpd.sjoin(
    crash_gdf,
    buffer_gdf_sub,
    predicate="within",
    how="inner"
).copy()

crash_camera["CRASH_DATE"] = pd.to_datetime(crash_camera["CRASH_DATE"], errors="coerce")
crash_camera["GO LIVE DATE"] = pd.to_datetime(crash_camera["GO LIVE DATE"], errors="coerce")
crash_camera = crash_camera.dropna(subset=["CRASH_DATE","GO LIVE DATE","INTERSECTION","CRASH_

crash_camera["crash_month"] = crash_camera["CRASH_DATE"].dt.to_period("M")
crash_camera["go_live_month"] = crash_camera["GO LIVE DATE"].dt.to_period("M")
crash_camera["event_time"] = (crash_camera["crash_month"] - crash_camera["go_live_month"]).ap

event_min, event_max = -24, 24
crash_camera = crash_camera[(crash_camera["event_time"] >= event_min) & (crash_camera["event_

# intersection × event_time
panel = (
    crash_camera.groupby(["INTERSECTION","event_time"])["CRASH_RECORD_ID"]
    .nunique()
    .reset_index(name="crash_count")
)


intersections = panel["INTERSECTION"].unique()
event_grid = np.arange(event_min, event_max + 1)
full_index = pd.MultiIndex.from_product([intersections, event_grid], names=["INTERSECTION","e

panel_full = (
    panel.set_index(["INTERSECTION","event_time"])[["crash_count"]]
    .reindex(full_index)
    .fillna({"crash_count": 0})
    .reset_index()
)
```

```
event_avg = (
    panel_full.groupby("event_time")["crash_count"]
    .mean()
    .reset_index()
)
```

```
import altair as alt
import pandas as pd

chart = alt.Chart(event_avg).mark_line(point=True).encode(
    x=alt.X("event_time:Q", title="Months Since Installation"),
    y=alt.Y("crash_count:Q",
            title="Avg Monthly Crashes per Intersection (within 200m)",
            scale=alt.Scale(zero=False)),
    tooltip=[
        alt.Tooltip("event_time:Q", title="Event time (months)"),
        alt.Tooltip("crash_count:Q", title="Avg crashes", format=".2f")
    ]
).properties(
    width=700,
    height=400,
    title=f"Event Study: Red Light Cameras (Install Year  {cutoff_year}, ±24 Months)"
)

vline = alt.Chart(pd.DataFrame({"x":[0]})).mark_rule(strokeDash=[6,4], color="red").encode(x=
chart + vline
```

```
alt.LayerChart(...)
```

```
OUT_PATH = DERIVED_DIR / "crashes_by_community_year_hour_type.csv"

def load_crashes(crash):
    usecols = ["CRASH_DATE", "LATITUDE", "LONGITUDE", "FIRST_CRASH_TYPE", "MOST_SEVERE_INJUR
    df = crash[usecols].copy()

    df["CRASH_DATE"] = pd.to_datetime(
        df["CRASH_DATE"], errors="coerce", infer_datetime_format=True
    )

    df = df.dropna(subset=["CRASH_DATE", "LATITUDE", "LONGITUDE"]).copy()
    df["LATITUDE"] = pd.to_numeric(df["LATITUDE"], errors="coerce")
```

```
    df["LONGITUDE"] = pd.to_numeric(df["LONGITUDE"], errors="coerce")
    df = df.dropna(subset=["LATITUDE", "LONGITUDE"]).copy()

    df["FIRST_CRASH_TYPE"] = (
        df["FIRST_CRASH_TYPE"].astype(str).str.strip().fillna("UNKNOWN")
    )

    df["MOST_SEVERE_INJURY"] = df["MOST_SEVERE_INJURY"].astype(str).str.strip()
    df["fatal_flag"] = (df["MOST_SEVERE_INJURY"] == "FATAL").astype(int)

    df["year"] = df["CRASH_DATE"].dt.year
    df["hour"] = df["CRASH_DATE"].dt.hour
    return df


def main():
    DERIVED_DIR.mkdir(parents=True, exist_ok=True)

    areas = community_boundaries()
    areas = areas[["COMMUNITY", "geometry"]]

    cr = load_crashes(crash)

    cr_gdf = gpd.GeoDataFrame(
        cr,
        geometry=gpd.points_from_xy(cr["LONGITUDE"], cr["LATITUDE"]),
        crs="EPSG:4326",
    )

    joined = gpd.sjoin(cr_gdf, areas, how="inner", predicate="within")

    agg = (
    joined.groupby(["COMMUNITY", "year", "hour", "FIRST_CRASH_TYPE"])
    .agg(
        crash_count=("fatal_flag", "size"),
        fatal_crashes=("fatal_flag", "sum")
    )
    .reset_index()
    )

    agg.to_csv(OUT_PATH, index=False)
```

```python
if __name__ == "__main__":
    main()
```